

MediaGuardX

Complete Technical Deep-Dive & Study Guide

Understand every component of the system.

Be ready for any technical question.

Version 2.0.0 | February 2026

Deepfake Detection Platform | Supabase Architecture

1. TRUST SCORE GENERATION — How is it calculated?

The trust score is a weighted composite score (0-100) combining 8 different analyzers. Think of it like a final exam where each subject has a different weight:

Analyzer	Weight	What it does
ML Model (EfficientNet-B0)	40%	Our trained neural network — primary detector
Sightengine API	20%	Third-party deepfake detection API
Fingerprint Analyzer	12%	Frequency-domain analysis (FFT) for GAN artifacts
Metadata Analyzer	8%	Checks EXIF data (camera, timestamps)
Compression Analyzer	8%	JPEG blocking artifacts, platform signatures
Audio Analyzer	4%	Voice cloning detection (spectral flatness, MFCC)
Emotion Analyzer	4%	Face-audio emotion mismatch
Sync Analyzer	4%	Lip-sync vs audio onset correlation

Formula

```
composite = sum(analyzer_score x weight) / total_weight
```

Penalty System

- If 3+ analyzers score below 70 → subtract 15 points
- If 2 analyzers score below 70 → subtract 8 points
- If neither ML model nor Sightengine is available → cap at 65

Insight

The system is designed so no single analyzer can dominate. Even if the ML model says "real" (score 100), if the metadata, fingerprint, and compression analyzers all flag problems, the multi-flag penalty drags the composite score down. This makes the system robust against adversarial attacks targeting just one detection method.

2. TRUST SCORE THRESHOLDS — What are they?

Score Range	Label	Meaning
70 – 100	Authentic	Likely genuine, trustworthy
40 – 69	Suspicious	Uncertain, needs investigation
0 – 39	Deepfake	Likely manipulated/AI-generated

Answer for your guide:

"We chose 70 as the authentic threshold because it means the majority of weighted analyzers agree the media is genuine. 40 is the lower boundary because below that, enough signals indicate manipulation to classify it as deepfake."

3. CAMERA TIMESTAMPS / EXIF — What happens when there are NO camera details?

This is the Metadata Analyzer (metadata_analyzer.py). Here's exactly what it does:

Step 1: Extract EXIF data using PIL

```
exif_data = img.getexif() # reads camera make, model, timestamps, GPS, etc.
```

Step 2: Check for camera info

Looks for fields: Make, Model, LensModel

Step 3: If NO EXIF data exists

- missingCamera = True
- Score penalty: -20 points (out of 100)
- This is a mild penalty because real images from the web (Unsplash, social media) also routinely lack EXIF

Step 4: Check timestamp consistency

- Compares DateTimeOriginal vs DateTime (modified date)
- If they differ → irregularTimestamps = True → -30 points

Step 5: Check compression ratio

```
ratio = file_size / (width x height x 3)
```

- If ratio > 0.8 (unusually low compression) → -25 points
- If ratio < 0.01 (extreme compression) → -25 points (re-encoding)

Step 6: Multi-flag penalty

- If 2+ flags set (missing camera + timestamps + compression) → extra -10 points

Answer for your guide:

"When an image has no camera metadata or timestamps — which is extremely common for any web-downloaded image — the metadata analyzer gives a moderate penalty of 20 points (out of its 100-point sub-score). But this only contributes 8% to the final trust score, so missing EXIF alone reduces the composite by only ~1.6 points. The system doesn't rely on metadata alone — it cross-references with the ML model (40%), Sightengine (20%), and frequency-domain analysis (12%) to make the final decision. Missing EXIF is suggestive but not conclusive."

4. ML MODEL — How does EfficientNet-B0 work?

Architecture

```

Input Image (224x224 RGB)
  |
  v
EfficientNet-B0 Backbone (pretrained feature extractor)
  |   Extracts 1280 visual features
  v
Dropout(0.3)
  v
Linear(1280 -> 128) + ReLU
  v
Dropout(0.2)
  v
Linear(128 -> 2)  <-- 2 classes: [fake, real]
  v
Softmax -> [P(fake), P(real)]
  
```

What it actually does

- Resizes image to 224x224
- Normalizes with ImageNet mean/std: [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
- Forward pass through the network → outputs 2 logits
- Softmax converts to probabilities: P(real) is used as the trust score

For video

Extracts up to 12 evenly-spaced frames, runs each through the model, and averages the real-probability across all frames.

Training Details

Parameter	Value
Base Architecture	EfficientNet-B0 (pretrained on ImageNet)
Training Dataset	200 images (100 fake + 100 real)
Fake Sources	StyleGAN via thispersondoesnotexist.com (1024x1024)
Real Sources	Royalty-free photographs from Unsplash (640x640)
Train/Val Split	80% training / 20% validation
Validation Accuracy	95%
Discrimination Gap	81.2% (fake avg 15.5% vs real avg 96.8%)
Real Recall	100% (zero false accusations on real images)
Optimizer	AdamW (lr=1e-4, weight_decay=1e-4)
Loss Function	CrossEntropyLoss with label smoothing (0.1)
Data Augmentation	Random crop, horizontal flip, rotation, color jitter

5. LIVE CAMERA — How is trust score generated?

The live camera uses a different, simpler pipeline than file upload detection:

```

+-----+      WebSocket      +-----+      HTTPS      +-----+
| Browser | =====> | Backend | =====> | Sightengine |
| Webcam  | Base64 JPEG   | Server  | Image bytes | API       |
| (5 FPS) | <=====      |         | <=====      |           |
|         | Trust Score JSON |         | Deepfake prob |           |
+-----+      +-----+      +-----+

```

Step by step

- Frontend captures webcam frame → converts to base64 JPEG
- Sends over WebSocket to backend
- Backend decodes base64 → raw image bytes
- Sends to Sightengine API (the only analyzer used for live frames)
- Sightengine returns deepfake_score (0 = authentic, 1 = deepfake)
- Conversion: $\text{trust_score} = (1 - \text{deepfake_score}) \times 100$
- Same thresholds: ≥ 70 = Authentic, 40-69 = Suspicious, < 40 = Deepfake

Rate limiting

- Max 5 frames/second (200ms interval)
- Frames larger than 1.5MB are rejected

Answer for your guide:

"In live camera mode, we send each webcam frame to the Sightengine deepfake detection API, which uses its own deep learning model trained on millions of images to return a deepfake probability. We convert that to our trust score: $(1 - \text{deepfake_probability}) \times 100$. We don't use the full 8-analyzer pipeline for live frames because the heuristic analyzers need a saved file and would add too much latency for real-time analysis."

Insight

The live camera pipeline is intentionally simpler — only Sightengine — because real-time analysis needs <200ms response time. The full 8-analyzer pipeline runs on file uploads where latency isn't critical. This is a deliberate engineering trade-off between accuracy and speed.

6. AI FILTERS — Will applying filters reduce the score?

Yes, potentially. Here's why:

The Sightengine API's deepfake model is trained to detect AI-manipulated faces. Filters like:

- Face smoothing / beauty mode
- Face reshape (bigger eyes, smaller nose)
- AR face overlays (dog ears, cartoon face)

These modify the pixel distribution of the face in ways similar to deepfake manipulation:

- Smoothing destroys natural skin texture → looks like GAN-generated skin
- Face reshape changes facial geometry → triggers the same features deepfake swaps change
- AR overlays blend artificial elements → creates blending artifacts similar to face-swap

For file upload detection (full pipeline), filters would also trigger:

- Fingerprint analyzer: Modified frequency spectrum (FFT detects unnatural patterns)
- Compression analyzer: Filter processing changes JPEG artifact patterns
- Metadata: No camera EXIF if saved from a filter app

Answer for your guide:

"Yes, AI filters will likely reduce the trust score because they introduce the same types of pixel-level modifications that deepfake tools create — smoothed textures, altered facial geometry, blending artifacts. Our ML model was trained to detect these exact patterns. A beauty filter that smooths skin will look statistically similar to StyleGAN-generated skin to our EfficientNet-B0 model. This is actually correct behavior — the system detects manipulation, not just deepfakes specifically."

7. FINGERPRINT ANALYZER — How does it identify the deepfake tool?

a) Frequency-Domain Analysis (FFT)

Image -> Grayscale -> 2D FFT -> Magnitude Spectrum -> Radial Profile -> Autocorrelation

- GAN-generated images have periodic peaks in the frequency spectrum (fingerprint of the neural network's upsampling layers)
- Natural images have smooth frequency falloff
- If >2 periodic peaks detected → high probability of GAN origin

b) Face Boundary Analysis

- Detect face using Haar Cascade
- Analyze the edge region around the face boundary
- Calculate Laplacian variance (blur detection)
- Low variance = face boundary is blurred = likely face-swap blending

c) GAN Dimension + Metadata Check

- Known GAN output sizes: 1024x1024, 512x512, 256x256
- Combined with no EXIF + high JPEG quality → strong GAN indicator

Known tools it can identify

Tool	Detection Signals
StyleGAN	Periodic spectral peaks + GAN dimensions + no EXIF
FaceSwap	Low high-frequency ratio + face boundary blur
DeepFaceLab	Face boundary blur + mask edge artifacts
NeuralTextures	High-frequency energy ratio > 0.7

8. AUDIO ANALYSIS — How does voice cloning detection work?

Uses librosa library to analyze 5 audio features:

Feature	What it measures	Cloned voice indicator
Spectral Flatness	How noise-like vs tonal	> 0.4 = too uniform (synthetic)
Zero-Crossing Rate (std)	Variation in waveform crossings	< 0.01 = unnaturally uniform
MFCC Variance	Mel-freq cepstral coefficients	< 5.0 = low variation (synthetic)
Pitch Stability (std)	How much the pitch varies	< 10 Hz = unnaturally stable
Spectral Rolloff	Where freq energy concentrates	< 2000 Hz = limited range

Clone Score Calculation

```
clone_score = flags x 20 + (spectral_flatness_bonus)
```

Higher = more likely cloned. Then inverted for trust: $\text{trust} = 100 - \text{clone_score}$

9. GRAD-CAM HEATMAP — What does the heatmap show?

Grad-CAM (Gradient-weighted Class Activation Mapping) shows which regions of the image the ML model focused on to make its decision.

How it works

- Forward pass through EfficientNet-B0
- Hook into last convolutional layer (`features[-1]`)
- Backward pass from the fake class score
- Compute gradient-weighted average of feature maps
- ReLU → normalize → resize to original image → overlay as color heatmap

Reading the heatmap

- Red/hot regions = areas the model thinks are most manipulated
- Blue/cold regions = areas the model thinks are authentic

The system also extracts XAI regions — bounding boxes around high-activation areas with descriptions like "High manipulation probability (85%) in Middle-Center area."

10. ADAPTIVE LEARNING — How does the system improve?

- Users can submit feedback on any detection: "this was actually real" or "this was actually fake"
- Feedback images are saved to adaptive_data/fake/ or adaptive_data/real/
- After 10+ feedback samples, admin can trigger retraining
- Retraining fine-tunes the existing model with very low learning rate (1e-5) for 5 epochs
- New model is hot-swapped in memory — no server restart needed

Quick Reference Card

Rapid answers for your guide's questions

Question	Short Answer
"How is trust score gene..	Weighted composite of 8 analyzers: ML model 40%, Sightengine API 20%, 6 heur..
"Is there a threshold?"	Yes: ≥ 70 = Authentic, 40-69 = Suspicious, < 40 = Deepfake
"No camera/timestamp on ..	Only 8% weight, -20pt penalty. System relies on ML model and frequency analy..
"How does live camera wo..	Each frame sent to Sightengine API, trust = $(1 - \text{deepfake_prob}) \times 100$, same ..
"Will AI filters reduce ..	Yes — filters modify pixel patterns similarly to deepfakes, model correctly ..
"What ML model?"	EfficientNet-B0 with custom classifier head, 95% val accuracy, trained on 20..
"What is Grad-CAM?"	Visualizes which image regions the model focused on — red = suspicious areas
"How does audio analysis..	5 features via librosa: spectral flatness, ZCR, MFCC, pitch stability, rolloff
"How does it identify th..	FFT frequency analysis + face boundary blur + GAN dimension check
"Can the model improve?"	Yes — adaptive learning: user feedback + auto-retrain after 10+ samples